

**Option #2: NLP Chatbot Project Data Collection**

Justan Dunn

Colorado State University Global

CSC525: Principles of Machine Learning

Module 4 Portfolio Milestone

Dong Nguyen

June 7, 2026

## **Option #2: NLP Chatbot Project Data Collection**

For the portfolio project, the selected path is Option #2, the NLP Chatbot Project. The proposed system is a closed-domain customer-service routing and support chatbot that could be adapted for companies in different industries. The chatbot's business purpose is to interpret a customer's natural-language message, identify the most likely service category, retrieve an approved response when one is available, and route low-confidence cases to a human agent. This milestone focuses on the data collection and training strategy because the value of the chatbot will depend less on a complicated interface and more on whether its data represents real customer language, reliable answer selection, and the specific policies or procedures of the organization using it.

The most appropriate data strategy is a hybrid approach that combines public research datasets with a small, controlled enterprise knowledge base. A public dataset can help the model learn general customer-service language patterns, but a company-specific dataset is still needed because customer-support answers must be tied to approved business rules. For that reason, the final project should not rely on an open-domain chat dataset such as movie dialogue or broad social media posts. Those datasets may be interesting for general conversation, but they are a weak fit for a business chatbot that needs accurate routing, traceable answers, and safe fallback behavior.

### **Dataset Selection Strategy**

The first selected dataset is the Relational Strategies in Customer Service corpus. Beaver et al. (2017) describe this dataset as a publicly available commercial customer-service corpus built from human interactions with intelligent virtual agents in travel and telecommunications. The dataset is useful for this project because customer messages often contain greetings, backstory, emotion, gratitude, frustration, and other relational language that may not directly identify the user's intent. For a routing chatbot, those segments matter because the model must recognize the core request even when it is mixed with conversational text.

RSiCS can therefore support preprocessing rules, intent-labeling examples, and tests for whether the chatbot can separate the customer's service need from the extra language around it.

The second selected dataset is the Microsoft Research WikiQA Corpus. WikiQA contains question and candidate-answer sentence pairs, including answer labels and examples where no candidate sentence is correct (Yang et al., 2015). This is important for the portfolio project because the chatbot should not always assume that the best available match is good enough to send to a customer. WikiQA can be used to train and test answer-selection logic by teaching the model to rank candidate responses and by supporting an answer-triggering threshold. In practical terms, WikiQA helps with the retrieval side of the chatbot: given a customer question, the model should identify whether a candidate answer is relevant enough to return.

The third dataset will be a custom, sanitized enterprise FAQ and standard operating procedure corpus. This corpus should contain approved categories such as billing, account access, order status, product troubleshooting, returns, service scheduling, and escalation. Each row should include a route label, sample customer question, approved answer, source document or policy owner, and optional keywords. This controlled dataset is necessary because public data can teach general language behavior, but it cannot safely define a company's actual return policy, account rules, service-level commitments, or escalation processes. The custom corpus also gives the final project a clear audit trail from each answer back to the business source that approved it.

### **Training Plan**

The initial model should be retrieval-based rather than generative. A retrieval-based design is more appropriate for the course project because it is easier to explain, test, and control. The first version will use Python, pandas, and scikit-learn. Text will be normalized by lowercasing, trimming whitespace, removing unnecessary punctuation, and combining the sample question, keywords, and route label into a searchable training field. The model will

then use a term frequency-inverse document frequency representation to convert the training text into weighted features. This design follows a standard information-retrieval approach in which documents and queries are represented in a vector space and then ranked by similarity (Manning et al., 2008).

In implementation, scikit-learn's TfidfVectorizer can convert the approved question and answer entries into a TF-IDF document-term matrix, and the fitted vectorizer can transform new customer inputs into the same feature space (scikit-learn developers, n.d.). The main tuning choices will include unigram and bigram features, document-frequency limits to reduce overly common terms, and a minimum similarity threshold for returning an answer. WikiQA will be used to test whether the retrieval logic can rank relevant answer sentences above irrelevant candidates. RSiCS will be used to test whether preprocessing and routing still work when customer messages include conversational or emotional language. The custom FAQ/SOP corpus will be split into training and validation examples so the final chatbot can be evaluated on company-specific questions it did not directly memorize.

Evaluation should include top-1 accuracy, top-3 accuracy, route-label accuracy, and fallback quality. Top-1 accuracy checks whether the first answer is correct. Top-3 accuracy checks whether the correct response appears among the best few candidates, which is helpful if the interface later asks the customer to choose from suggested topics. Fallback quality checks whether the chatbot refuses to answer when similarity is too low. This is a key technical consideration because an enterprise customer-service chatbot should make uncertainty visible instead of producing unsupported answers.

### **Chatbot Use of the Training**

At runtime, the chatbot will apply the same preprocessing steps used during training. A customer message will be normalized, transformed into the fitted TF-IDF vector space, and compared with the stored FAQ/SOP entries. If the best match exceeds the confidence threshold, the bot will return the approved answer and route label. If the score is moderate, the

bot can ask a clarifying question or present the closest few categories. If the score is low, the bot will route the conversation to a human agent or provide a safe fallback such as asking the customer to rephrase the request. This workflow lets the chatbot use training data to control both answer selection and escalation behavior.

The design also supports later improvement. Logged customer questions can be reviewed, deduplicated, sanitized, and added to the custom corpus only after approval. That process prevents the model from learning private or incorrect responses directly from production traffic. It also gives the company a practical feedback loop: common fallback questions become candidates for new FAQ entries, confusing categories can be rewritten, and weak routing labels can be corrected. This makes the chatbot useful as a managed business system rather than a one-time class demonstration.

### **Conclusion**

The recommended dataset plan for the Module 8 portfolio project is to use RSiCS for customer-service language patterns, WikiQA for answer-selection and threshold training, and a sanitized enterprise FAQ/SOP corpus for the final domain-specific knowledge base. This combination meets the technical needs of a practical customer-service chatbot while keeping the project explainable, testable, and appropriate for business use. Most importantly, it separates general language learning from approved company answers, which is necessary for a chatbot that could be implemented safely across different organizations.

## References

- Beaver, I., Freeman, C., & Mueen, A. (2017). An annotated corpus of relational strategies in customer service. arXiv. <https://doi.org/10.48550/arXiv.1708.05449>
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to information retrieval. Cambridge University Press. <https://nlp.stanford.edu/IR-book/>
- scikit-learn developers. (n.d.). TfidfVectorizer. scikit-learn documentation. [https://scikit-learn.org/stable/modules/generated/sklearn.feature\\_extraction.text.TfidfVectorizer.html](https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html)
- Yang, Y., Yih, W.-t., & Meek, C. (2015). WikiQA: A challenge dataset for open-domain question answering. In L. Marquez, C. Callison-Burch, & J. Su (Eds.), Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (pp. 2013-2018). Association for Computational Linguistics. <https://doi.org/10.18653/v1/D15-1237>